

OFFICEPULSE

Full System Architecture, API and File Structure Documentation

React + Bun + Redis + Neon

Project definition

A real-time office energy digital twin for 15 devices across three rooms, with PIR-based occupancy, smart alerts, safe automation, an agentic AI dashboard chatbot, and a Discord bot that all use one shared backend and one source of truth.

Hackathon Preliminary Round - Engineering Design Specification

Document version: 1.0

Prepared: 3 July 2026

Contents

This document is organized as an implementation-ready specification. The section numbers are stable references for the team, README, issues, and demo script.

1. Purpose, Scope, and Decisions
2. Requirements and Feature Scope
3. Architecture and Component Responsibilities
4. Monorepo and File Structure
5. Domain Model and Neon Database Design
6. Redis Design
7. API Conventions
8. Complete REST API Catalog
9. WebSocket Realtime API
10. Agentic AI and Shared Tool Contracts
11. Discord Bot Interface
12. Simulator, PIR, and Hardware Gateway
13. Alert and Automation Rules
14. Energy, Cost, Savings, and Carbon Calculations
15. Frontend Architecture and UX
16. Security and Authorization
17. Configuration and Environment Variables
18. Local Development and Scripts
19. Testing and Validation
20. Reliability, Observability, and Fail-safe Behavior
21. Deployment and CI/CD
22. Implementation Roadmap
23. Acceptance Criteria
24. Demo Plan
25. References

1. Purpose, Scope, and Decisions

This specification converts the hackathon case into a coherent software architecture and an implementation contract. It covers the repository layout, data ownership, APIs, realtime events, automation, AI tools, Discord integration, persistence, testing, and deployment.

1.1 Fixed project assumptions

- Exactly 15 electrical devices are monitored: three rooms, each containing two fans and three lights.
- The rooms are Drawing Room, Work Room 1, and Work Room 2.
- Each room uses one PIR motion sensor for privacy-friendly occupancy detection; the system does not identify people or store images.
- The hackathon demo uses simulated device state and power data. The ESP32/PIR design is represented as a realistic integration path.
- Neon Postgres is the durable system of record for configuration, history, alerts, actions, and analytics.
- Redis holds hot state, event streams, realtime fan-out, locks, rate limits, short-lived sessions, and alert deduplication state.
- The React dashboard, dashboard chatbot, Discord bot, simulator, and hardware gateway use the same Bun backend.
- The dashboard is protected by a 6-digit platform PIN. Sensitive actions such as device control, rule changes, emergency shutdown, and AI actions also require recent PIN re-verification.
- Full Admin/Operator/Viewer role-based access and per-room QR codes are excluded from the main scope because they do not improve the core judging criteria.

Engineering priority

Finish the mandatory realtime monitoring, power calculation, alerts, Discord commands, diagrams, and documentation before enabling advanced automation or analytics. Standout features should extend the same data model rather than create disconnected demos.

1.2 Technology baseline

Area	Decision
Dashboard	React + TypeScript + Vite, built and run with Bun; responsive PWA and public-display mode.
Backend	Bun runtime using Bun.serve for REST and WebSocket endpoints; modular domain services.
Realtime	Bun WebSockets, Redis Pub/Sub for low-latency fan-out, and Redis Streams for recoverable asynchronous processing.
Database	Neon managed Postgres using pooled runtime connections and SQL migrations.
Cache and messaging	Redis for current snapshots, streams, pub/sub, locks, dedupe, rate limits, and sessions.
Discord	Discord REST, Interactions, and Gateway integration. Required prefix commands plus recommended slash-command equivalents.
AI	Provider adapter with OpenAI Responses API as the reference implementation; structured tool calling and explicit confirmations for actions.
Testing	Bun test for unit/integration tests, browser E2E tests for dashboard behavior, contract tests for APIs and events.

2. Requirements and Feature Scope

The MVP reproduces every required behavior from the case. The improvised scope adds operational value while preserving a clear delivery order.

2.1 Mandatory MVP features

- **15-device monitoring:** Monitor six fans and nine lights across the three rooms.
- **Device state:** Store and display each device name, room, ON/OFF state, current power draw, rated wattage, and last-changed timestamp.
- **Dynamic simulation:** Continuously update device states and telemetry so the system always has live data.
- **Shared backend:** Keep the dashboard and Discord bot synchronized through one backend and one source of truth.
- **Realtime dashboard:** Update devices, power, alerts, and timestamps without a manual refresh.
- **Room grouping:** Organize all devices and room totals by Drawing Room, Work Room 1, and Work Room 2.
- **Power meter:** Display total office power in watts and a per-room breakdown.
- **Daily energy estimate:** Calculate the current day energy usage in kWh.
- **After-hours detection:** Alert when devices remain ON outside 9:00 AM to 5:00 PM.
- **Long-running room detection:** Alert when all five devices in one room remain ON continuously for over two hours.
- **Active alert panel:** Display timestamped active anomalies and their affected room or device.
- **Discord status command:** Implement !status using live backend data.
- **Discord room command:** Implement !room <name> using live backend data.
- **Discord usage command:** Implement !usage with current watts and daily kWh.
- **Humanized responses:** Turn verified backend facts into concise, friendly Discord responses.
- **Public repository:** Include a clear README, diagrams, setup steps, environment variables, commands, and demo instructions.
- **Three-minute demo:** Show realtime updates, Discord commands, alert behavior, architecture, and the representative circuit.

2.2 Standout improvised features

- **Platform PIN access gate:** Require a valid 6-digit PIN before any dashboard, room, energy, alert, report, or AI-chat data is displayed.
- **Interactive digital twin:** Use the office top-view as the main monitoring interface.
- **Animated devices:** Rotate active fans and make active lights glow.
- **Clickable details:** Show live watts, ON duration, last change, energy, alert status, and recent activity for each device.
- **PIR occupancy:** Use one PIR sensor per room and track recent movement.
- **Occupancy confidence:** Show Occupied, Recently Active, Vacant, or Unknown with the time since last motion.
- **Vacant-room waste:** Alert when devices remain ON after a room becomes vacant.
- **Privacy-friendly occupancy:** Use motion only; do not store images, identities, or people counts.
- **Agentic AI dashboard chatbot:** Give the dashboard the same conversational capabilities as the Discord bot.
- **Shared AI tools:** Make the dashboard assistant and Discord bot use the same verified backend tools and calculations.
- **Natural-language questions:** Answer questions about status, waste, alerts, savings, rankings, forecasts, and closing tasks.
- **Inactivity auto-shutdown:** Warn, wait for a grace period, and then shut down room devices after no PIR motion.
- **Scheduled shutdown:** Apply office-closing, weekend, and holiday shutdown rules.
- **Device timer:** Set a countdown before a selected fan or light turns OFF.
- **Room and office shutdown:** Support one-action room shutdown and full-office shutdown.
- **Emergency shutdown:** Provide a protected immediate shutdown command for all controllable loads.
- **Manual safety override:** Temporarily disable automation during overtime, meetings, or maintenance.
- **Warning, cancel, and snooze:** Warn before automatic shutdown and allow authorized cancellation or delay.
- **Energy-saving mode:** Keep only configured essential devices active when occupancy is low.
- **Room presets:** Apply Working, Meeting, Lunch Break, Cleaning, Overtime, and Closing modes.
- **Smart closing checklist:** Show exactly which rooms and devices still need attention before leaving.
- **Peak-power alert:** Warn when total or room power exceeds a configured threshold.
- **Abnormal wattage detection:** Flag devices drawing significantly more or less than expected.
- **Rapid-switching detection:** Detect repeated ON/OFF changes that may indicate a fault.
- **Device health monitoring:** Detect stale ESP32, PIR, simulator, or device telemetry.
- **Maintenance reminders:** Recommend inspection using operating hours, abnormal power, or switching behavior.
- **Alert severity:** Classify alerts as information, warning, or critical.
- **Alert lifecycle:** Support acknowledgement, resolution, mute, snooze, deduplication, and escalation.
- **Audit history:** Record every automatic and manual action with actor, reason, timestamp, and result.
- **Historical charts:** Show office, room, and device energy over time.

- **Live activity timeline:** Show state changes, occupancy, alerts, AI actions, and shutdown events.
- **Usage rankings:** Rank rooms and devices by energy, runtime, and waste.
- **Room efficiency score:** Summarize room performance using transparent factors.
- **Cost and waste analysis:** Estimate daily cost, after-hours waste, and projected monthly bill.
- **Savings report:** Calculate energy and money saved by automated shutdowns.
- **Weekly comparison:** Compare this week with the previous week.
- **Energy budgets:** Track daily and monthly targets and show progress.
- **Carbon estimate:** Convert consumed and saved energy into configurable estimated emissions.
- **Downloadable reports:** Export usage, alerts, actions, savings, and comparisons as CSV or PDF.
- **Mobile-responsive PWA:** Make the full dashboard installable and usable on phones and tablets.
- **Public display mode:** Provide a simplified read-only screen for an office monitor or television through a PIN-authenticated kiosk session.
- **Real-time notification preferences:** Let users choose Discord alert categories and severity thresholds.
- **Fail-safe display:** Preserve and label the last known state during temporary backend or network failure.
- **Backup local control:** Let the ESP32 follow basic safe shutdown rules when the backend is unavailable.
- **Demo scenarios:** Trigger normal work, lunch, after-hours waste, sensor failure, peak load, and vacancy scenarios.
- **Accelerated time:** Demonstrate two-hour and closing-time rules in seconds.
- **System status:** Show backend, database, Redis, simulator, AI, Discord, and hardware connectivity.

2.3 Explicitly deferred or excluded

- Full Admin, Operator, and Viewer role hierarchy: replaced with one platform PIN gate plus recent PIN re-verification for sensitive controls.
- Per-room QR codes: optional only after the core system is complete; they add little value in a three-room office.
- Real mains-voltage switching in the hackathon demo: use a safe low-voltage representative circuit and simulated commands.
- Camera-based occupancy and face recognition: unnecessary, privacy-invasive, and outside the case.

3. Architecture and Component Responsibilities

The architecture separates durable truth, hot state, domain logic, asynchronous processing, realtime delivery, and user interfaces. Every interface consumes the same domain services.

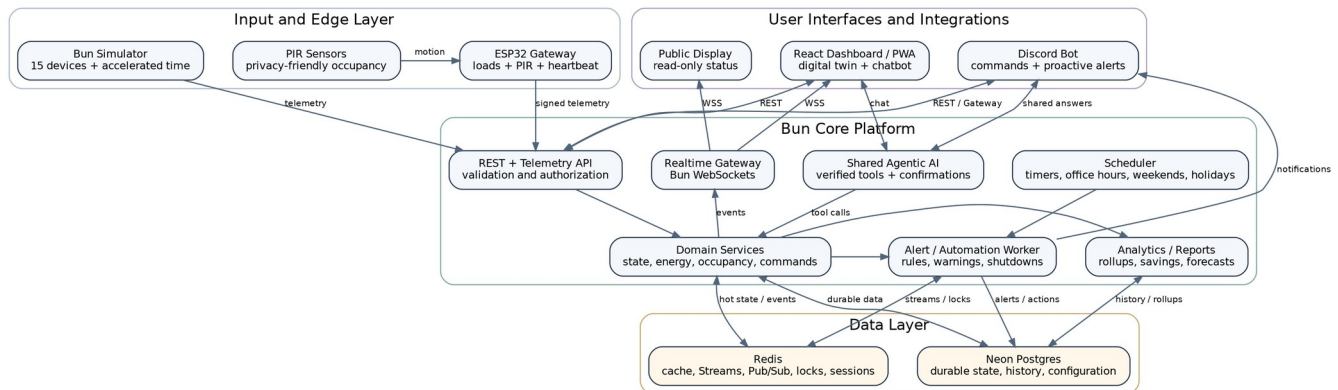


Figure 1. High-level system architecture. Diagram created with Graphviz; no Mermaid is used.

3.1 Architecture principles

- Neon owns durable truth; Redis accelerates delivery but is never the only copy of important history.
- All device commands pass through one command service so dashboard, AI, Discord, schedules, and fail-safe logic cannot conflict silently.
- Events use stable schemas from the shared contracts package.
- Read operations may use cached snapshots; writes are validated, authorized, audited, and persisted.
- AI never invents device facts or calculations. It calls backend tools and then explains verified results.
- Actions proposed by AI require a confirmation step unless they are read-only.
- WebSocket loss does not corrupt state; clients request a fresh snapshot when reconnecting.
- Automations use distributed locks and idempotency keys to avoid duplicate shutdowns.

- The simulator and real ESP32 gateway use the same telemetry contract.

3.2 Component responsibilities

Component	Primary responsibility
React dashboard	Platform PIN access gate, digital twin, live cards, charts, alerts, closing checklist, chatbot, responsive PWA, public display, and control confirmation dialogs.
Bun API server	REST routing, validation, authorization, snapshots, commands, WebSocket upgrades, and integration endpoints.
Domain services	Device state, occupancy, energy calculations, alert decisions, schedules, presets, actions, reporting, and audit records.
Simulator	Deterministic device/PIR telemetry, scenario control, accelerated time, and fault injection.
Discord bot	Required prefix commands, slash commands, proactive alerts, preferences, and shared AI answers.
AI agent service	Tool selection, structured arguments, conversation state, verified read tools, proposed actions, and explicit confirmation handling.
Alert/automation worker	Rule evaluation, dedupe, escalation, warnings, safe shutdown execution, retries, and notifications.
Redis	Cache, Streams, Pub/Sub, locks, rate limits, temporary sessions, AI short-term state, and alert dedupe.
Neon Postgres	Rooms, devices, current/durable history, alerts, actions, rules, schedules, budgets, analytics, conversations, and configuration.
ESP32 gateway	PIR and switch telemetry, relay output for representative loads, local fail-safe rules, and heartbeat.

3.3 Realtime update path

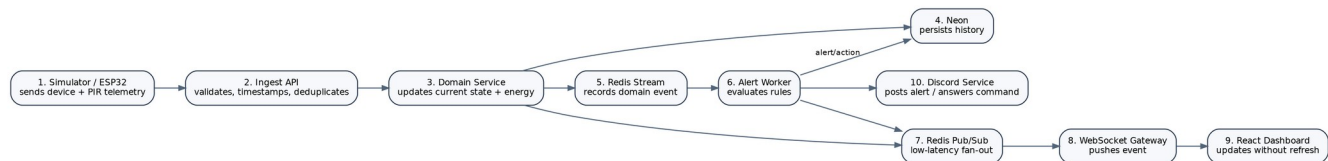


Figure 2. Device-to-dashboard and Discord event flow.

4. Monorepo and File Structure

Use a Bun workspace monorepo so all applications share contracts, validation, database utilities, event types, and configuration. Commit bun.lock to keep installs reproducible.

```

officepulse/
├── apps/
│   ├── dashboard/                # React + Vite PWA
│   │   ├── public/
│   │   │   ├── icons/
│   │   │   └── manifest.webmanifest
│   │   └── src/
│   │       ├── app/              # router, providers, error boundaries
│   │       ├── pages/            # overview, room, analytics, alerts, settings
│   │       └── features/
│   │           ├── digital-twin/
│   │           ├── devices/
│   │           ├── occupancy/
│   │           ├── energy/
│   │           ├── alerts/
│   │           ├── automations/
│   │           ├── closing-checklist/
│   │           ├── agent-chat/
│   │           └── public-display/
  
```

components/	
hooks/	
lib/	# api client, ws client, formatting
stores/	# small UI-only state
styles/	
main.tsx	
index.html	
vite.config.ts	
package.json	
api/	# Bun HTTP + WebSocket server
src/	
server.ts	
routes/	# auth, bootstrap, domain APIs
middleware/	# platform session, rate limits, CSRF
websocket/	
controllers/	
bootstrap.ts	
package.json	
worker/	# alerts, automations, reports, rollups
src/	
consumers/	
jobs/	
schedulers/	
worker.ts	
package.json	
simulator/	# deterministic dummy telemetry
src/scenarios/	
src/engine/	
src/cli.ts	
package.json	
discord-bot/	
src/commands/	
src/events/	
src/embeds/	
src/notifications/	
src/bot.ts	
package.json	
packages/	
auth/	# PIN hashing, Redis sessions, cookie guards
contracts/	# API schemas, event types, enums
domain/	# business logic and calculations
db/	# Bun SQL client, repositories, migrations
redis/	# keys, streams, pub/sub, locks
agent/	# AI provider adapter and tool registry
config/	# typed environment configuration
logger/	# structured logs and correlation IDs
ui/	# shared React components
testing/	# fixtures, fake clocks, test helpers
database/	
migrations/	
seeds/	
schema.sql	
docs/	
diagrams/	
openapi/openapi.yaml	
websocket-events.md	
alert-rules.md	
hardware-pin-map.md	
demo-script.md	
adr/	# architecture decision records
infra/	
docker-compose.yml	
Dockerfile.api	
Dockerfile.worker	
Dockerfile.bot	
deployment/	
scripts/	
dev-all.ts	
migrate.ts	
seed.ts	
register-discord-commands.ts	
generate-openapi.ts	
tests/	
contract/	
integration/	
e2e/	
load/	
.env.example	
.gitignore	
biome.json	
bunfig.toml	
package.json	
tsconfig.base.json	
bun.lock	
README.md	

4.1 Package boundaries

Boundary	Rule
packages/contracts	Contains only serializable schemas and types. It must not import database or UI code.
packages/domain	Contains pure rules and calculations. It accepts repositories/interfaces instead of direct network clients.
packages/db	Owns SQL, migrations, transactions, repository implementations, and Neon connection handling.
packages/redis	Owns key names, stream groups, publish/subscribe helpers, locks, dedupe, and cache TTL policy.
packages/agent	Owns AI provider abstraction, prompts, tool schemas, tool dispatcher, action confirmation state, and channel adapters.
apps/api	Maps HTTP and WebSocket requests to domain use cases. No business rules should live inside route handlers.
apps/worker	Consumes streams and executes asynchronous work. It is independently restartable.
apps/dashboard	Never connects directly to Redis, Neon, Discord, or the AI provider; it only calls the backend.

4.2 Recommended root scripts

Command	Purpose
bun run dev	Run dashboard, API, worker, bot, and simulator concurrently.
bun run dev:api	Run API with watch mode.
bun run dev:dashboard	Run Vite dashboard.
bun run dev:worker	Run stream consumers and schedulers.
bun run dev:bot	Run Discord integration.
bun run simulate	Run the default deterministic office scenario.
bun run db:migrate	Apply SQL migrations using the direct Neon connection.
bun run db:seed	Create rooms, 15 devices, default rules, schedules, and demo settings.
bun run test	Run Bun unit and integration tests.
bun run test:e2e	Run browser end-to-end tests.
bun run typecheck	Type-check every workspace.
bun run lint	Run formatter/linter checks.
bun run build	Build production artifacts.
bun run discord:register	Register slash commands for the configured guild or globally.
bun run openapi:generate	Generate or validate the OpenAPI document from shared contracts.

5. Domain Model and Neon Database Design

Neon stores configuration and history, while Redis mirrors hot state. The schema supports both the mandatory demo and the improved analytics and automation features.

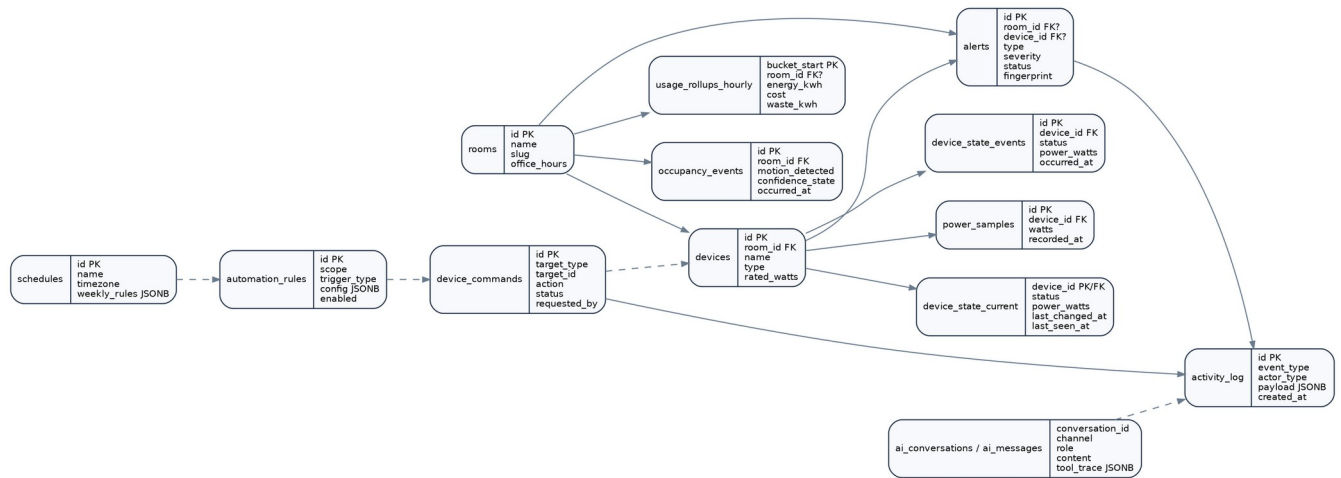


Figure 3. Simplified entity relationship view. Detailed columns are defined below.

5.1 Core domain identifiers

```

type RoomSlug = "drawing" | "work1" | "work2";
type DeviceType = "fan" | "light";
type DeviceStatus = "on" | "off" | "unknown";
type OccupancyState = "occupied" | "recently_active" | "vacant" | "unknown";
type AlertSeverity = "info" | "warning" | "critical";
type AlertStatus = "active" | "acknowledged" | "snoozed" | "resolved";
type CommandStatus = "pending" | "warning" | "confirmed" | "executing" |
    "succeeded" | "failed" | "cancelled";

```

5.2 Required tables

Table	Purpose
rooms	Three fixed rooms, timezone, office hours, occupancy timeout, peak threshold, and public display order.
devices	Fifteen fixed device records with room, type, label, rated watts, hardware channel, and essential-device flag.
device_state_current	One current row per device: status, measured watts, source, last changed, last seen, and version.
device_state_events	Append-only device state changes for timelines, runtime, and audit calculations.
power_samples	Periodic measured or simulated watt samples for charts and energy integration.
occupancy_state_current	Current occupancy confidence and last motion for each room.
occupancy_events	Motion events, state transitions, source, and timestamp.
alerts	Active and historical alerts with type, severity, fingerprint, timestamps, context, and lifecycle status.
alert_events	Acknowledgement, snooze, escalation, resolution, and notification history.
device_commands	Every manual, AI, schedule, preset, emergency, or fail-safe command and execution result.
automation_rules	Trigger, conditions, scope, grace period, warning policy, action, enabled state, and priority.
automation_executions	Individual automation decisions and their warning, confirmation, execution, and outcome timeline.
schedules	Weekly office hours, weekend rules, timezone, and closing policy.

Table	Purpose
holidays	Date-specific closures or custom hours.
room_presets	Working, Meeting, Lunch, Cleaning, Overtime, Closing, and Energy-saving target states.
usage_rollups_hourly	Hourly energy, cost, after-hours waste, occupied waste, and peak values by room and office.
energy_budgets	Daily and monthly target kWh or cost limits.
maintenance_counters	Runtime hours, switch cycles, abnormal readings, next service threshold, and status.
notification_preferences	Discord destination, enabled alert types, minimum severity, quiet hours, and summaries.
discord_installations	Guild, channel, application configuration, and encrypted integration metadata.
ai_conversations	Conversation owner/channel, title, status, and timestamps.
ai_messages	User, assistant, and tool messages with structured tool traces and action confirmation references.
activity_log	Unified human-readable timeline for the dashboard and audit.
system_components	Heartbeat and health state for API, worker, simulator, bot, Redis, Neon, and ESP32 gateways.
app_settings	Versioned global settings such as tariff, carbon factor, currency, default timeouts, and demo mode.

5.3 Important database constraints and indexes

- Unique room slug and unique device name within each room.
- CHECK constraints for status, type, severity, command action, and non-negative wattage.
- Unique active alert fingerprint where status is not resolved, preventing duplicate active alerts.
- Composite indexes on device_state_events(device_id, occurred_at DESC) and power_samples(device_id, recorded_at DESC).
- Composite index on alerts(status, severity, created_at DESC).
- Composite index on occupancy_events(room_id, occurred_at DESC).
- Composite index on usage_rollups_hourly(bucket_start, room_id).
- JSONB only for flexible rule context and tool traces; frequently filtered values should have normal columns.
- All timestamps use timestampz and the application stores UTC while presenting Asia/Dhaka local time.
- Use transaction boundaries for command creation, state transition, audit entry, and outbox/event publication metadata.

5.4 Connection policy

- Use a pooled Neon connection URL for runtime API, bot, and workers.
- Use a direct Neon connection URL for migrations and maintenance operations that require session behavior.
- Keep connection creation in packages/db; no application should create ad hoc database clients.
- Use short transactions and parameterized SQL.
- Create separate Neon branches for development, staging, and pull-request testing when practical.

6. Redis Design

Redis is used deliberately for speed and coordination. Durable history remains in Neon. Pub/Sub is low latency but at-most-once, so recoverable work uses Streams and consumer groups.

6.1 Key and stream naming

Redis key/stream	Purpose
cache:office:snapshot	JSON snapshot used by dashboard bootstrap; TTL 3-5 seconds.

Redis key/stream	Purpose
state:device:{deviceId}	Hash with current status, watts, last changed, last seen, and version.
state:occupancy:{roomId}	Hash with occupancy state, last motion, confidence, and source.
stream:telemetry	Validated device/PIR telemetry awaiting domain processing.
stream:domain-events	Recoverable ordered device, occupancy, alert, and action events.
stream:automation	Scheduled and rule-triggered automation jobs.
stream:reports	Long-running report generation jobs.
pubsub:realtime	Best-effort fan-out to WebSocket gateways.
dedupe:alert:{fingerprint}	Short-lived alert dedupe marker.
lock:automation:{scope}	Distributed lock preventing duplicate room/office actions.
session:control:{tokenHash}	Short-lived protected-control session.
confirm:ai-action:{id}	Pending AI action and confirmation deadline.
rate:{route}:{subject}	Sliding-window or token-bucket rate limiting.
health:component:{id}	Recent component heartbeat and metadata.
ws:resume:{clientId}	Optional last event ID for reconnect assistance.
cache:analytics:{queryHash}	Short-lived cached analytics response.

6.2 Stream consumer groups

Consumer group	Responsibility
telemetry-domain	Consumes stream:telemetry and updates current state, Neon history, and domain events.
alert-engine	Consumes stream:domain-events and evaluates alert rules.
automation-engine	Consumes stream:automation and executes warning, delay, confirmation, and shutdown workflows.
analytics-rollup	Consumes domain events and writes hourly rollups.
discord-notifier	Consumes relevant alert and summary events and posts to Discord.
report-generator	Consumes stream:reports and creates CSV/PDF exports.

6.3 Cache rules

- Cache only derived reads and current snapshots; never treat a cache hit as proof that a command executed.
- Invalidate or replace room and office snapshots immediately after a committed state change.
- Include a state version and generated_at timestamp in every snapshot.
- If Redis is unavailable, read from Neon and continue with reduced realtime performance.
- If Neon is unavailable, show last-known Redis data as stale and disable non-emergency writes.

7. API Conventions

All browser-facing APIs are versioned under `/api/v1`. Except for health checks and PIN-session endpoints, they require a valid platform session. Internal telemetry endpoints are isolated under `/internal` and require a service secret or signed request.

7.1 Base URLs and transports

```

REST API:      https://<host>/api/v1
WebSocket:     wss://<host>/ws
Discord webhook/interactions: https://<host>/integrations/discord/interactions
Internal telemetry: https://<host>/internal/v1/telemetry

```

Health: <https://<host>/health> and [/ready](https://<host>/ready)

7.2 Standard response envelope

```
{
  "data": { ... },
  "meta": {
    "requestId": "req_01J...",
    "generatedAt": "2026-07-03T12:00:00.000Z",
    "stateVersion": 1842
  }
}
```

7.3 Standard error envelope

```
{
  "error": {
    "code": "CONTROL_AUTH_REQUIRED",
    "message": "Protected control authorization is required.",
    "details": { "action": "office.shutdown" },
    "requestId": "req_01J..."
  }
}
```

7.4 Cross-cutting API rules

- Use ISO 8601 UTC timestamps in payloads and include timezone-aware display values only as presentation metadata.
- Use stable string IDs or UUIDs; never expose database sequence assumptions.
- Validate every payload against shared schemas from packages/contracts.
- Use Idempotency-Key for commands, timers, report requests, and external telemetry batches.
- Use X-Request-Id or generate one; include it in logs and responses.
- Use cursor pagination for histories and activity streams.
- Return 202 Accepted for asynchronous actions and include an execution ID.
- Return 409 Conflict for state-version conflicts, duplicate timers, or automation locks.
- Return 423 Locked when a safety override or emergency state prevents the requested operation.
- All dashboard read endpoints require a valid platform PIN session. Control endpoints additionally require recent PIN re-verification; unauthenticated requests return 401 PLATFORM_AUTH_REQUIRED.
- Internal endpoints require HMAC signatures or service tokens and are never exposed to the browser.

7.5 Common query parameters

Parameter	Meaning
from / to	ISO timestamps defining a reporting interval.
granularity	minute, hour, day, or week.
roomId / deviceId	Optional filters.
status / severity / type	Alert and action filters.
cursor / limit	Cursor pagination; default 50, maximum 200.
include	Optional related resources such as devices, alerts, or occupancy.
timezone	Presentation timezone; default Asia/Dhaka.
fresh	When true, bypass analytics cache for administrative diagnostics.

8. Complete REST API Catalog

The following endpoints cover the MVP and every retained improvised feature. The exact implementation may combine closely related routes, but all capabilities need an equivalent contract.

8.1 System, health, and bootstrap

Method	Path	Purpose	Access
GET	/health	Liveness check for the API process.	Public
GET	/ready	Checks Neon, Redis, migrations, and critical dependencies.	Public
GET	/api/v1/bootstrap	Returns office snapshot, rooms, devices, occupancy, alerts, permissions, and realtime config.	Platform PIN
GET	/api/v1/system/components	Returns health and last heartbeat for API, worker, simulator, bot, AI, and gateways.	Platform PIN
GET	/api/v1/config/public	Returns non-secret UI configuration, tariff label, timezone, and feature flags.	Platform PIN
GET	/api/v1/activity	Returns paginated office activity timeline.	Platform PIN

8.2 Rooms and digital twin

Method	Path	Purpose	Access
GET	/api/v1/rooms	Lists all rooms with live summary.	Platform PIN
GET	/api/v1/rooms/{roomId}	Returns one room, devices, occupancy, power, alerts, and active preset.	Platform PIN
GET	/api/v1/rooms/{roomId}/snapshot	Returns a compact realtime room snapshot.	Platform PIN
GET	/api/v1/rooms/{roomId}/devices	Lists room devices.	Platform PIN
GET	/api/v1/rooms/{roomId}/activity	Returns room-specific timeline.	Platform PIN
GET	/api/v1/rooms/{roomId}/efficiency	Returns efficiency score and explainable factors.	Platform PIN
GET	/api/v1/rooms/{roomId}/closing-checklist	Returns devices and alerts requiring action before leaving.	Platform PIN
GET	/api/v1/office/closing-checklist	Returns the complete office closing checklist.	Platform PIN

8.3 Devices and commands

Method	Path	Purpose	Access
GET	/api/v1/devices	Lists all 15 devices with live state.	Platform PIN
GET	/api/v1/devices/{deviceId}	Returns device details, health, runtime, usage, and alerts.	Platform PIN
GET	/api/v1/devices/{deviceId}/history	Returns state change history.	Platform PIN
GET	/api/v1/devices/{deviceId}/usage	Returns device energy, cost, runtime, and waste.	Platform PIN
GET	/api/v1/devices/{deviceId}/	Returns maintenance counters and	Platform PIN

Method	Path	Purpose	Access
	maintenance	recommendations.	
POST	/api/v1/devices/{deviceId}/commands	Creates ON/OFF command.	Control
POST	/api/v1/devices/{deviceId}/timers	Creates countdown timer.	Control
GET	/api/v1/devices/{deviceId}/timers	Lists active and recent timers.	Platform PIN
DELETE	/api/v1/devices/{deviceId}/timers/{timerId}	Cancels a pending timer.	Control
POST	/api/v1/rooms/{roomId}/commands/shutdown	Creates room-wide shutdown.	Control
POST	/api/v1/office/commands/shutdown	Creates full-office shutdown.	Control
POST	/api/v1/office/commands/emergency-shutdown	Creates immediate protected emergency shutdown.	Control + confirm
GET	/api/v1/commands/{commandId}	Returns command and execution status.	Platform PIN
POST	/api/v1/commands/{commandId}/cancel	Cancels a pending or warning-stage command.	Control

8.4 Occupancy and PIR

Method	Path	Purpose	Access
GET	/api/v1/occupancy	Returns live occupancy for all rooms.	Platform PIN
GET	/api/v1/rooms/{roomId}/occupancy	Returns state, last motion, confidence, and timeout.	Platform PIN
GET	/api/v1/rooms/{roomId}/occupancy/history	Returns motion and occupancy transitions.	Platform PIN
PUT	/api/v1/rooms/{roomId}/occupancy/settings	Updates inactivity timeout and warning grace.	Control
POST	/internal/v1/telemetry/occupancy	Ingests signed PIR motion telemetry.	Service

8.5 Live energy and analytics

Method	Path	Purpose	Access
GET	/api/v1/energy/live	Returns current total and room watts.	Platform PIN
GET	/api/v1/energy/today	Returns today kWh, cost, waste, peak, and savings.	Platform PIN
GET	/api/v1/energy/history	Returns time-series energy and power.	Platform PIN
GET	/api/v1/energy/comparison/weekly	Compares current and previous week.	Platform PIN
GET	/api/v1/energy/rankings	Ranks rooms/devices by energy, runtime, and waste.	Platform PIN
GET	/api/v1/energy/waste/after-hours	Returns after-hours energy and cost.	Platform PIN
GET	/api/v1/energy/savings	Returns automation savings with evidence.	Platform PIN
GET	/api/v1/energy/carbon	Returns configurable emissions estimate.	Platform PIN

Method	Path	Purpose	Access
GET	/api/v1/energy/forecast/bill	Returns month-end bill forecast and confidence.	Platform PIN
GET	/api/v1/energy/budgets	Returns daily/monthly targets and progress.	Platform PIN
PUT	/api/v1/energy/budgets/{budgetId}	Creates or updates a target.	Control

8.6 Alerts and alert rules

Method	Path	Purpose	Access
GET	/api/v1/alerts	Lists alerts by status, type, severity, room, or device.	Platform PIN
GET	/api/v1/alerts/summary	Returns counts and highest-priority alerts.	Platform PIN
GET	/api/v1/alerts/{alertId}	Returns full alert context and history.	Platform PIN
POST	/api/v1/alerts/{alertId}/acknowledge	Acknowledges an alert.	Control
POST	/api/v1/alerts/{alertId}/resolve	Manually resolves an alert when allowed.	Control
POST	/api/v1/alerts/{alertId}/snooze	Snoozes an alert until a timestamp.	Control
POST	/api/v1/alerts/{alertId}/mute	Mutes a non-critical alert category temporarily.	Control
GET	/api/v1/alert-rules	Lists configured rules.	Platform PIN
GET	/api/v1/alert-rules/{ruleId}	Returns one rule.	Platform PIN
PUT	/api/v1/alert-rules/{ruleId}	Updates threshold, scope, severity, or escalation.	Control
POST	/api/v1/alert-rules/{ruleId}/test	Evaluates a rule against a supplied safe test snapshot.	Control

8.7 Automations, schedules, presets, and overrides

Method	Path	Purpose	Access
GET	/api/v1/automations	Lists automation rules and status.	Platform PIN
POST	/api/v1/automations	Creates a rule.	Control
PUT	/api/v1/automations/{automationId}	Updates a rule.	Control
DELETE	/api/v1/automations/{automationId}	Deletes a non-system rule.	Control
POST	/api/v1/automations/{automationId}/enable	Enables a rule.	Control
POST	/api/v1/automations/{automationId}/disable	Disables a rule.	Control
GET	/api/v1/automation-executions	Lists warnings, confirmations, and executions.	Platform PIN
GET	/api/v1/automation-executions/{executionId}	Returns execution timeline.	Platform PIN
POST	/api/v1/automation-executions/{executionId}/cancel	Cancels a pending execution.	Control
POST	/api/v1/automation-executions/{executionId}/snooze	Delays an execution.	Control
GET	/api/v1/schedules	Returns weekly, weekend, and	Platform PIN

Method	Path	Purpose	Access
		closing schedules.	
PUT	/api/v1/schedules/{scheduleId}	Updates schedule.	Control
GET	/api/v1/holidays	Lists closure and custom-hour dates.	Platform PIN
POST	/api/v1/holidays	Adds holiday/custom hours.	Control
DELETE	/api/v1/holidays/{holidayId}	Removes holiday override.	Control
GET	/api/v1/presets	Lists room presets.	Platform PIN
POST	/api/v1/rooms/{roomId}/presets/{presetId}/apply	Applies a room preset.	Control
GET	/api/v1/manual-overrides	Lists active safety overrides.	Platform PIN
POST	/api/v1/manual-overrides	Creates a time-limited override.	Control
DELETE	/api/v1/manual-overrides/{overrideId}	Ends an override.	Control

8.8 Reports and exports

Method	Path	Purpose	Access
POST	/api/v1/reports	Queues CSV or PDF report generation.	Control
GET	/api/v1/reports	Lists generated reports.	Platform PIN
GET	/api/v1/reports/{reportId}	Returns report status and metadata.	Platform PIN
GET	/api/v1/reports/{reportId}/download	Downloads completed report.	Platform PIN
DELETE	/api/v1/reports/{reportId}	Deletes report metadata/file reference.	Control

8.9 Agentic AI chatbot

Method	Path	Purpose	Access
POST	/api/v1/ai/conversations	Creates dashboard or Discord conversation.	Platform PIN
GET	/api/v1/ai/conversations/{conversationId}	Returns conversation and tool trace summaries.	Platform PIN
POST	/api/v1/ai/conversations/{conversationId}/messages	Sends user message and returns streamed or final answer.	Platform PIN
POST	/api/v1/ai/actions/{actionId}/confirm	Confirms a proposed protected action.	Control + confirm
POST	/api/v1/ai/actions/{actionId}/cancel	Cancels a proposed action.	Platform PIN
GET	/api/v1/ai/actions/{actionId}	Returns proposed action, expiry, and status.	Platform PIN
DELETE	/api/v1/ai/conversations/{conversationId}	Deletes conversation from the application.	Control

8.10 Discord integration

Method	Path	Purpose	Access
POST	/integrations/discord/interactions	Receives and verifies Discord interaction requests.	Discord signed

Method	Path	Purpose	Access
POST	/internal/v1/discord/notify	Queues proactive alert or summary message.	Service
GET	/api/v1/integrations/discord	Returns non-secret installation/channel status.	Platform PIN
PUT	/api/v1/integrations/discord/preferences	Updates channel and notification preferences.	Control
POST	/api/v1/integrations/discord/test	Sends a test message.	Control

8.11 Simulator and hardware telemetry

Method	Path	Purpose	Access
POST	/internal/v1/telemetry/devices	Ingests batch device state and watts.	Service
POST	/internal/v1/telemetry/occupancy	Ingests PIR motion telemetry.	Service
POST	/internal/v1/components/{componentId}/heartbeat	Updates component health.	Service
POST	/internal/v1/simulator/scenarios/{scenarioId}/start	Starts deterministic demo scenario.	Service/control
POST	/internal/v1/simulator/stop	Stops active scenario.	Service/control
PUT	/internal/v1/simulator/time-scale	Sets accelerated simulation time.	Service/control
POST	/internal/v1/simulator/tick	Advances deterministic simulation manually.	Service/control
GET	/api/v1/simulator/status	Returns active scenario and simulated clock.	Platform PIN
GET	/api/v1/simulator/scenarios	Lists available demo scenarios.	Platform PIN

8.12 Platform access, protected control authorization, and settings

Method	Path	Purpose	Access
POST	/api/v1/auth/pin/verify	Verifies the 6-digit platform PIN and creates a short-lived authenticated dashboard session.	Public
GET	/api/v1/auth/session	Returns whether a valid platform session exists and its expiry metadata.	Public
POST	/api/v1/auth/session/refresh	Refreshes an active platform session within the configured policy.	Platform PIN
DELETE	/api/v1/auth/session	Logs out and revokes the current platform session.	Platform PIN
POST	/api/v1/control-auth/verify	Re-verifies the same platform PIN and starts a short recent-control authorization window.	Platform PIN
POST	/api/v1/control-auth/refresh	Refreshes an active recent-control authorization window.	Control
DELETE	/api/v1/control-auth/session	Ends the recent-control authorization window without logging out of the platform.	Control
GET	/api/v1/settings	Returns configurable operational settings.	Platform PIN

Method	Path	Purpose	Access
PUT	/api/v1/settings	Updates tariff, carbon factor, thresholds, and defaults.	Control
GET	/api/v1/notification-preferences	Returns Discord notification preferences.	Platform PIN
PUT	/api/v1/notification-preferences	Updates notification types, severity, and quiet hours.	Control

8.13 Important request and response examples

Device command request:

```
POST /api/v1/devices/fan-work1-1/commands
Idempotency-Key: 7f56f9ab-...
Content-Type: application/json

{
  "action": "off",
  "reason": "Manual dashboard control",
  "expectedStateVersion": 1842
}
```

Accepted command response:

```
{
  "data": {
    "commandId": "cmd_01J...",
    "status": "pending",
    "target": { "type": "device", "id": "fan-work1-1" },
    "action": "off"
  },
  "meta": { "requestId": "req_01J...", "generatedAt": "2026-07-03T12:00:00Z" }
}
```

PIR telemetry request:

```
POST /internal/v1/telemetry/occupancy
X-Device-Id: esp32-work1
X-Timestamp: 1783080000
X-Signature: <HMAC-SHA256>

{
  "eventId": "occ_evt_01J...",
  "roomId": "work1",
  "motionDetected": true,
  "sensorId": "pir-work1-1",
  "observedAt": "2026-07-03T12:00:00Z"
}
```

9. WebSocket Realtime API

After platform PIN authentication, the dashboard opens one WebSocket connection using the authenticated session cookie. The server validates the session, sends compact domain events, and clients fall back to /bootstrap after reconnect or version gaps.

9.1 Connection and subscription

```
Client -> Server
{
  "type": "subscribe",
  "topics": ["office", "rooms:work1", "alerts", "system"],
  "lastEventId": "evt_01J..."
}
```

```

Server -> Client
{
  "type": "subscribed",
  "connectionId": "ws_01J...",
  "stateVersion": 1842,
  "heartbeatSeconds": 25
}

```

9.2 Server event envelope

```

{
  "eventId": "evt_01J...",
  "type": "device.state.changed",
  "occurredAt": "2026-07-03T12:00:00Z",
  "stateVersion": 1843,
  "scope": { "officeId": "office-1", "roomId": "work1", "deviceId": "fan-work1-1" },
  "payload": { "status": "off", "powerWatts": 0, "lastChangedAt": "..." }
}

```

9.3 Event types

Event	Purpose
office.snapshot.updated	Compact aggregate power, active counts, occupancy, and alert summary.
device.state.changed	ON/OFF or last-changed transition.
device.power.updated	Measured/simulated watt change without a state transition.
device.health.changed	Online, stale, disconnected, or fault state.
occupancy.motion.detected	Raw recent motion event.
occupancy.state.changed	Occupied, recently active, vacant, or unknown transition.
energy.live.updated	Office and room watt totals.
energy.budget.updated	Budget progress changed.
alert.created	New deduplicated alert.
alert.updated	Acknowledged, snoozed, escalated, or changed.
alert.resolved	Condition cleared or manually resolved.
automation.warning	Upcoming automatic action and cancellation deadline.
automation.executing	Automatic action has started.
automation.completed	Automatic action result and savings evidence.
command.updated	Manual or AI command status changed.
activity.created	New timeline item.
system.component.changed	API, Redis, Neon, bot, simulator, AI, or ESP32 health changed.
simulator.clock.updated	Accelerated demo time changed.
report.completed	CSV/PDF report is ready.

9.4 Reconnect rules

- Client uses exponential backoff with jitter and a maximum delay.
- Client sends its last event ID and state version when reconnecting.
- If the server cannot replay the gap, it responds with `resync_required` and the client calls `/api/v1/bootstrap`.
- UI shows a stale banner and the last trusted update time while disconnected.
- Commands are disabled while realtime state is stale unless emergency policy explicitly permits them.

10. Agentic AI and Shared Tool Contracts

The dashboard chatbot and Discord bot use one agent service. The model can reason about verified data through tools, but it cannot directly access Neon, Redis, or device channels.

Agent safety rule

Read tools may execute immediately. Any tool that changes a device, timer, preset, schedule, alert state, override, or shutdown must create a proposed action and wait for explicit user confirmation.

10.1 Agent execution loop

1. Receive a user message with channel context: dashboard or Discord.
2. Load a small conversation summary and the current public office snapshot.
3. Ask the model to choose from registered function tools using strict structured arguments.
4. Validate tool arguments with shared schemas.
5. Execute read tools or create a pending proposed action for protected tools.
6. Return tool results to the model for a concise, human-friendly answer.
7. Persist the user message, tool trace summary, answer, and any proposed action ID.
8. Use the same answer formatting facts across dashboard and Discord, with channel-specific presentation.

10.2 Read-only tools

Tool	Returns
get_office_snapshot	Current devices, room watts, occupancy, alerts, and freshness.
get_room_status	One room with device states, occupancy, runtime, power, and alerts.
get_device_details	Device health, state, power, history, and maintenance information.
get_usage_summary	Current watts, selected period kWh, cost, waste, and savings.
compare_usage	Current period versus previous period.
get_usage_rankings	Top rooms/devices by consumption, runtime, or waste.
list_active_alerts	Active alerts filtered by severity, type, or room.
explain_alert	Verified reason, evidence, and recommended response.
get_closing_checklist	Outstanding rooms, devices, alerts, and pending shutdowns.
get_energy_budget_progress	Target, actual, projection, and variance.
get_bill_forecast	Month-end forecast with assumptions and confidence.
get_system_health	Component health and last heartbeat.
get_maintenance_recommendations	Devices requiring inspection and reasons.

10.3 Protected action tools

Tool	Action
propose_device_command	Turn one device ON or OFF after confirmation.
propose_device_timer	Create an OFF countdown after confirmation.
propose_room_shutdown	Switch off all controllable room devices after confirmation.
propose_office_shutdown	Switch off all devices after confirmation.
propose_emergency_shutdown	Immediate protected office shutdown with a stronger confirmation phrase.

Tool	Action
propose_room_preset	Apply Working, Meeting, Lunch, Cleaning, Overtime, Closing, or Energy-saving preset.
propose_alert_acknowledgement	Acknowledge selected alert.
propose_alert_snooze	Snooze a non-critical alert until a specified time.
propose_automation_cancel	Cancel a pending automatic shutdown.
propose_manual_override	Create a time-limited automation override.
propose_schedule_change	Change closing, weekend, or holiday automation.
propose_budget_change	Update daily or monthly energy budget.

10.4 Example agent action contract

```
{
  "tool": "propose_room_shutdown",
  "arguments": {
    "roomId": "work2",
    "reason": "Closing checklist shows all five devices still on",
    "warningSeconds": 60
  },
  "result": {
    "actionId": "ai_action_01J...",
    "status": "awaiting_confirmation",
    "expiresAt": "2026-07-03T12:02:00Z",
    "confirmationText": "Confirm shutdown of all five devices in Work Room 2"
  }
}
```

10.5 AI reliability rules

- The model never calculates energy from prose; it requests calculated results from tools.
- Tool schemas use strict enums for room, device, action, period, and severity.
- The final answer includes data freshness when a user asks for live state.
- If tools fail, the assistant states that the live system could not be verified.
- No action executes from natural language alone; confirmation is bound to a specific action ID and expiry.
- Prompt injection inside device labels, activity notes, or external messages is treated as untrusted data.
- Conversation retention is limited and configurable; secrets are never added to prompts.

11. Discord Bot Interface

The bot must answer from the same backend used by the dashboard. Prefix commands satisfy the case; slash commands improve usability and avoid relying only on message parsing.

11.1 Required commands

Command	Behavior
!status	Summarize all rooms, active devices, occupancy, total watts, and urgent alerts.
!room <drawing work1 work2>	Show one room with fans, lights, occupancy, watts, and alerts.
!usage	Show current power and today estimated kWh, cost, and waste.
/status	Slash-command equivalent of !status.
/room room:<name>	Slash-command equivalent of !room.
/usage period:<today week month>	Expanded usage query.
/alerts severity:<optional>	Show active alerts.
/closing	Show smart closing checklist.

Command	Behavior
/ask question:<text>	Use the shared agentic AI service.
/timer device:<id> duration:<minutes>	Propose a protected timer action.
/shutdown scope:<room office>	Propose a protected shutdown action.
/snooze alert:<id> duration:<minutes>	Propose alert snooze.

11.2 Discord implementation notes

- Prefix commands require Gateway events and message content access; keep them for evaluation compatibility.
- Register slash commands because they provide native arguments, validation, and interaction responses.
- Use rich embeds for room status, alerts, closing summaries, and savings reports.
- Use deferred interaction responses when AI or analytics may exceed the immediate response window.
- Verify interaction signatures before processing HTTP interactions.
- Publish proactive alerts only after deduplication and notification-preference checks.
- Critical unresolved alerts may escalate according to the rule configuration.
- Dashboard and Discord answers should match because they call the same API and AI tools.

12. Simulator, PIR, and Hardware Gateway

The simulator creates live demo data for all 15 devices. The representative hardware path uses ESP32, PIR, low-voltage loads, and safe driver stages for one room.

12.1 Device inventory

Room	Device ID	Label	Type	Reference power
Drawing Room	drawing-fan-1	Fan 1	fan	60 W nominal
Drawing Room	drawing-fan-2	Fan 2	fan	60 W nominal
Drawing Room	drawing-light-1	Light 1	light	15 W nominal
Drawing Room	drawing-light-2	Light 2	light	15 W nominal
Drawing Room	drawing-light-3	Light 3	light	15 W nominal
Work Room 1	work1-fan-1	Fan 1	fan	60 W nominal
Work Room 1	work1-fan-2	Fan 2	fan	60 W nominal
Work Room 1	work1-light-1	Light 1	light	15 W nominal
Work Room 1	work1-light-2	Light 2	light	15 W nominal
Work Room 1	work1-light-3	Light 3	light	15 W nominal
Work Room 2	work2-fan-1	Fan 1	fan	60 W nominal
Work Room 2	work2-fan-2	Fan 2	fan	60 W nominal
Work Room 2	work2-light-1	Light 1	light	15 W nominal
Work Room 2	work2-light-2	Light 2	light	15 W nominal
Work Room 2	work2-light-3	Light 3	light	15 W nominal

12.2 Simulator scenarios

Scenario	Purpose
normal-day	Natural device changes and occupancy during office hours.
lunch-break	Rooms become vacant while selected devices remain ON.
after-hours-waste	Simulated clock moves past 5 PM with devices still ON.

Scenario	Purpose
all-on-two-hours	All five devices in a room remain ON long enough to trigger the mandatory rule.
peak-load	Many devices turn ON and total power crosses threshold.
abnormal-fan	One fan reports wattage above or below its normal tolerance.
rapid-switching	A device toggles repeatedly.
pir-silent	PIR stops reporting and component health becomes stale.
gateway-offline	ESP32 heartbeat stops.
closing-demo	Closing checklist, warning, cancellation, and eventual shutdown.
energy-saving	Low occupancy applies an essential-device preset.
all-off	Clean baseline with zero active load.

12.3 PIR occupancy logic

- Motion event immediately marks the room Occupied and updates last_motion_at.
- After a short recent-activity threshold, the room becomes Recently Active.
- After the configured inactivity timeout, the room becomes Vacant.
- If PIR or gateway data is stale, occupancy becomes Unknown rather than Vacant.
- Auto-shutdown requires Vacant state, fresh sensor health, no manual override, and no configured protected/essential device exception.
- Before shutdown, send a dashboard and Discord warning and start a grace countdown.
- Any new motion cancels the pending inactivity shutdown automatically.

12.4 Representative ESP32 pin map

Signal	ESP32 pin	Electrical note
PIR OUT	GPIO 27	Digital input, use stable 3.3 V-compatible output.
Fan 1 driver/relay	GPIO 16	Output through transistor/relay module; never drive load directly.
Fan 2 driver/relay	GPIO 17	Output through transistor/relay module.
Light 1 driver/relay	GPIO 18	Output through transistor/relay module.
Light 2 driver/relay	GPIO 19	Output through transistor/relay module.
Light 3 driver/relay	GPIO 21	Output through transistor/relay module.
Current sensor analog	GPIO 34	Input-only ADC pin; scale output safely to ESP32 range.
Status LED	GPIO 2	Gateway health indicator.
Ground	GND	Common ground for low-voltage simulation components.
Power	5 V / 3.3 V as required	Use appropriate regulated supplies and respect module voltage requirements.

Electrical safety

For the hackathon, use LEDs or low-voltage motors as representative loads. A real AC implementation requires correctly rated relays/contactors, isolation, fusing, enclosure, earthing, and qualified electrical review.

13. Alert and Automation Rules

Rules are configuration records evaluated against normalized device, occupancy, time, health, and energy facts.

Rule	Condition	Severity	Default response
after_hours_device_on	Device is ON outside configured office hours.	warning	Notify; optionally schedule closing shutdown.
all_room_devices_on_long	All five devices in a room are ON continuously for more than two hours.	warning	Notify and recommend room review.
vacant_room_devices_on	Room is Vacant and one or more devices remain ON beyond grace period.	warning	Warn, then optional inactivity shutdown.
peak_power	Office or room watts exceed configured threshold.	warning/critical	Notify; optionally apply energy-saving preset.
abnormal_wattage	Measured watts fall outside configured tolerance while ON.	warning	Recommend device inspection.
rapid_switching	Switch count exceeds threshold in a rolling window.	warning	Disable automation for device and inspect.
stale_device	Device telemetry last_seen exceeds threshold.	warning	Mark state Unknown and notify.
stale_pir	PIR/gateway heartbeat exceeds threshold.	critical for automation	Disable occupancy auto-shutdown.
budget_near_limit	Usage reaches configured percentage of daily/monthly budget.	info/warning	Notify and recommend saving actions.
forecast_over_budget	Projected bill or kWh exceeds target.	warning	Recommend schedule/preset adjustment.
unresolved_escalation	Critical alert remains active beyond escalation interval.	critical	Send repeated Discord escalation with cap.
maintenance_due	Runtime/switch cycles/abnormal events exceed maintenance threshold.	info/warning	Create maintenance recommendation.

13.1 Safe auto-shutdown state machine

```

IDLE
-> CONDITION_MET
-> WARNING_SENT
-> GRACE_PERIOD
    -> CANCELLED (motion / user cancel / override / stale sensor)
    -> CONFIRMED_BY_POLICY
-> EXECUTING
-> SUCCEEDED | FAILED
-> AUDITED + SAVINGS_EVIDENCE

```

13.2 Automation guard conditions

- Rule is enabled and schedule permits execution.
- PIR and device telemetry are fresh enough for the rule.
- No active manual override blocks the action.
- No higher-priority emergency or maintenance lock conflicts.
- No duplicate automation lock exists for the same scope.
- Essential-device policy permits the target devices to turn OFF.
- Grace period has expired without motion or cancellation.
- Expected state version still matches; otherwise re-evaluate.
- Command execution is idempotent and audit logging succeeds.

14. Energy, Cost, Savings, and Carbon Calculations

Calculations are deterministic backend functions. The AI assistant reads their results; it does not perform authoritative arithmetic itself.

Metric	Calculation
Current office power	Sum current power_watts for all 15 devices.
Room power	Sum current power_watts for devices in the room.
Interval energy	Integrate power over time: kWh = sum(watts x elapsed_hours) / 1000.
Estimated cost	energy_kwh x configured tariff_per_kwh.
After-hours waste	Energy consumed while outside office schedule, optionally excluding approved overtime.
Vacancy waste	Energy consumed while occupancy is Vacant and sensors are fresh.
Automation savings	Counterfactual energy avoided from action time until next natural/manual state change or bounded estimate window.
Weekly comparison	(current_week - previous_week) / previous_week x 100, with zero-baseline handling.
Bill forecast	Month-to-date actual plus forecast based on recent daily profile and remaining scheduled days.
Carbon estimate	energy_kwh x configurable kgCO2e_per_kwh factor.
Efficiency score	Transparent weighted score from waste ratio, after-hours usage, unresolved alerts, and normalized energy.
Maintenance runtime	Sum ON durations from state events; reconcile against current open interval.

14.1 Calculation quality rules

- Use event timestamps, not polling count, to calculate runtime and energy.
- Persist tariff and carbon-factor versions so historical reports remain reproducible.
- Label savings and bill forecasts as estimates and expose assumptions.
- Exclude stale or unknown readings from confident metrics and report data quality.
- Use hourly rollups for long periods and raw samples for short detailed charts.
- Tests must cover state transitions across midnight, weekends, holidays, and accelerated demo time.

15. Frontend Architecture and UX

The React dashboard should make live status understandable within seconds and keep advanced controls behind clear confirmation steps.

Screen	Main content
PIN access	6-digit PIN entry, validation feedback, temporary lockout countdown, session expiry handling, and no office data before authentication.
Overview	Office digital twin, total watts, room cards, occupancy, urgent alerts, system freshness, and chatbot.
Room detail	Five devices, motion status, power chart, active preset, timers, alerts, and room shutdown.
Analytics	History, comparisons, rankings, budgets, cost, forecast, carbon, and savings.
Alerts	Filters, evidence, severity, acknowledgement, snooze, resolution, and escalation history.
Automations	Inactivity rules, schedules, holidays, presets, warnings, overrides, and execution history.
Closing checklist	Outstanding devices, rooms, alerts, pending timers, and one-action safe shutdown.
Activity	Realtime timeline with device, motion, alert, command, AI, and system events.

Screen	Main content
Reports	Generate and download CSV/PDF reports.
Settings	Tariff, carbon factor, thresholds, timeouts, notification preferences, and protected PIN verification.
Public display	PIN-authenticated kiosk view with large read-only totals, room status, alerts, last update, and no sensitive controls.

15.1 Client data strategy

- Call /bootstrap once, then apply WebSocket events by state version.
- Use a server-state query library or small query layer for caching, retries, and invalidation.
- Keep only ephemeral UI state in local stores; backend state stays authoritative.
- Use optimistic UI only for acknowledgement/snooze where rollback is safe; device commands show Pending until confirmed by backend.
- Show live, stale, reconnecting, and offline states explicitly.
- Persist PWA shell assets, not sensitive live data, in the service worker cache.
- Public display mode must never expose control buttons or AI action confirmation.

15.2 Digital twin visual rules

- Fan animation speed indicates ON state, not exact RPM unless measured.
- Light glow indicates ON state; unknown state uses a distinct neutral warning style.
- Room border or badge shows occupancy confidence and last motion.
- Room power and active-device count remain visible without opening a details panel.
- Every animation has a reduced-motion alternative for accessibility.
- Color is never the only state indicator; use labels and icons.

16. Security and Authorization

The hackathon does not need enterprise identity management, but the platform must not expose office data without authentication. A simple PIN gate protects the dashboard, while recent re-verification protects actions that change devices or rules.

16.1 Platform PIN access and protected controls

- The application first shows a PIN entry screen; no room, device, energy, alert, report, or AI-chat data is loaded before authentication.
- Use one 6-digit platform PIN for the hackathon; sensitive controls require recent re-verification of the same PIN.
- Store only an Argon2id hash of the PIN in secure server configuration or the database; never send the hash to the React client.
- On successful login, issue a short-lived HttpOnly, Secure, SameSite cookie backed by Redis; enforce idle and absolute expiry and support explicit logout.
- Require recent PIN re-verification for emergency shutdown, device control, automation changes, schedule changes, and other sensitive actions.
- Rate-limit PIN attempts, temporarily lock repeated failures, and audit attempts without recording the PIN.
- Discord protected actions require an allowlist of user IDs plus an explicit confirmation interaction.

16.2 API and integration protections

Control	Implementation
Validation	Schema-validate all REST, WebSocket, Discord, AI-tool, and telemetry payloads.
Telemetry signing	ESP32/internal clients send timestamped HMAC signatures; reject replayed or old requests.
Discord verification	Verify Discord interaction signatures and protect the bot token.

Control	Implementation
Secrets	Use environment variables or a secret manager; never expose secrets to Vite client variables.
CORS	Allow only configured dashboard origins.
CSRF	Protect cookie-authenticated platform and control endpoints with SameSite cookies and a CSRF strategy where needed.
Rate limits	Limit platform PIN attempts, control re-verification, chat, reports, commands, and internal ingest abuse.
SQL safety	Use parameterized queries and least-privilege database credentials.
AI safety	Strict tools, untrusted-data boundaries, confirmation tokens, action expiry, and audit.
Logging	Redact tokens, PINs, signatures, and full AI prompts where they may contain secrets.

17. Configuration and Environment Variables

Every environment variable belongs to the backend, worker, bot, simulator, or dashboard build. Only variables prefixed for Vite are exposed to the browser.

```
# Core
NODE_ENV=development
APP_ENV=local
APP_TIMEZONE=Asia/Dhaka
API_PORT=3000
PUBLIC_API_URL=http://localhost:3000
PUBLIC_WS_URL=ws://localhost:3000/ws
CORS_ORIGINS=http://localhost:5173

# Neon
DATABASE_URL=postgresql://...-pooler.../officepulse?sslmode=require
DATABASE_DIRECT_URL=postgresql://.../officepulse?sslmode=require

# Redis
REDIS_URL=redis://localhost:6379
REDIS_PREFIX=officepulse

# Platform PIN access and control re-verification
PLATFORM_PIN_HASH=<argon2id-hash>
PLATFORM_SESSION_TTL_SECONDS=28800
PLATFORM_IDLE_TIMEOUT_SECONDS=1800
CONTROL_REAUTH_TTL_SECONDS=300
SESSION_COOKIE_SECRET=<secret>

# Discord
DISCORD_APPLICATION_ID=
DISCORD_PUBLIC_KEY=
DISCORD_BOT_TOKEN=
DISCORD_GUILD_ID=
DISCORD_ALERT_CHANNEL_ID=
DISCORD_ALLOWED_CONTROL_USER_IDS=

# AI provider
AI_PROVIDER=openai
OPENAI_API_KEY=
OPENAI_MODEL=<configured-current-model>
AI_MAX_TOOL_ROUNDS=6
AI_CONVERSATION_TTL_DAYS=30

# Energy and rules
DEFAULT_TARIFF_PER_KWH=
CURRENCY=BDT
CARBON_KG_PER_KWH=
OFFICE_OPEN_TIME=09:00
OFFICE_CLOSE_TIME=17:00
PIR_RECENT_SECONDS=120
PIR_VACANT_SECONDS=900
SHUTDOWN_WARNING_SECONDS=120
TELEMETRY_STALE_SECONDS=90
```

```
# Internal services
INTERNAL_SERVICE_TOKEN=
TELEMETRY_HMAC_SECRET=
SIMULATOR_TIME_SCALE=1
```

17.1 Feature flags

- FEATURE_AI_CHAT
- FEATURE_AUTO_SHUTDOWN
- FEATURE_PUBLIC_DISPLAY
- FEATURE_REPORTS
- FEATURE_PWA
- FEATURE CARBON
- FEATURE_BILL_FORECAST
- FEATURE_LOCAL_GATEWAY
- FEATURE_DEMO_CONTROLS

18. Local Development and Scripts

Local development should start with one command and provide deterministic seeded data. Redis can run in Docker, while Neon remains the shared managed Postgres environment or a development branch.

18.1 First-time setup

1. Install Bun and clone the public repository.
2. Copy .env.example to .env and fill Neon, Redis, Discord, and optional AI variables.
3. Run bun install.
4. Start Redis through docker compose up -d redis.
5. Run bun run db:migrate.
6. Run bun run db:seed to create the three rooms, 15 devices, schedules, rules, and presets.
7. Run bun run discord:register for slash commands.
8. Run bun run dev.
9. Open the dashboard and start a simulator scenario.

18.2 Seed guarantees

- Creates exactly three rooms and exactly 15 devices.
- Uses two fans at 60 W nominal and three lights at 15 W nominal per room.
- Creates default 9 AM-5 PM schedule and Asia/Dhaka timezone.
- Creates default mandatory alert rules and disabled advanced auto-shutdown until explicitly enabled.
- Creates all room presets and a safe default closing preset.
- Creates deterministic IDs so demo scripts and tests remain stable.

19. Testing and Validation

Testing focuses on correctness of state, energy, alerts, automation safety, realtime consistency, and failure behavior.

Test level	Coverage
Unit	Energy integration, cost, waste, efficiency score, occupancy transitions, rule predicates, and confirmation expiry.
Repository	SQL queries, constraints, transactions, and migrations against a Neon test branch.
Redis integration	Streams, consumer acknowledgements, dedupe, locks, cache invalidation, and Pub/Sub fan-out.

Test level	Coverage
API contract	Every endpoint status code, schema, platform PIN/session rule, control re-verification, idempotency behavior, and error envelope.
WebSocket contract	Subscription, ordering, state version, reconnect, stale state, and resync.
Discord	Prefix parsing, slash arguments, interaction signature handling, embeds, and proactive alerts.
AI tools	Strict argument validation, read-tool facts, protected action confirmation, expiry, and tool failure response.
Automation	Warning, motion cancellation, snooze, override, stale PIR block, distributed lock, retry, and audit.
E2E	PIN login, failed-attempt lockout, logout/session expiry, dashboard route protection, realtime state, digital twin animation, chatbot, alert lifecycle, timer, shutdown warning, and report download.
Failure	Redis loss, Neon loss, WebSocket disconnect, stale ESP32, bot outage, duplicate telemetry, and worker restart.
Load	Frequent telemetry updates, multiple WebSocket clients, and Discord/AI rate limiting.
Accessibility	Keyboard navigation, readable status labels, reduced motion, contrast, and screen-reader text.

19.1 Mandatory validation scenarios

- A device state change appears on dashboard without refresh and is returned by Discord immediately afterward.
- Total office watts equal the sum of all current device watts.
- Room totals equal the sum of their five devices.
- After-hours alert appears when simulated time passes 5 PM.
- All-devices-on alert appears after two accelerated hours.
- New PIR motion cancels a pending inactivity shutdown.
- Stale PIR prevents occupancy-based auto-shutdown.
- Duplicate telemetry and retry do not create duplicate state events or commands.
- AI gives the same verified status facts as !status.
- AI cannot execute a shutdown without confirmation.
- Redis restart does not lose durable alert/action history.
- Neon interruption produces stale read-only mode rather than false live state.

20. Reliability, Observability, and Fail-safe Behavior

The demo should visibly distinguish live, stale, and unavailable states. Silent failure is unacceptable in an energy-control system.

Concern	Design
Structured logs	JSON logs with timestamp, level, component, requestId, eventId, commandId, roomId, and deviceId.
Metrics	Telemetry rate, WebSocket clients, API latency, stream lag, alert count, command success, stale components, AI/tool latency.
Health	Liveness and readiness endpoints plus component heartbeat table and Redis keys.
Tracing	Correlation ID from telemetry or user request through domain event, alert, command, Discord, and WebSocket.
Error reporting	Capture unhandled errors with redacted context.
Audit	Persistent action and configuration change record.
Data freshness	Every snapshot and answer includes generated_at and source freshness.
Backpressure	Bound stream retention, consumer group monitoring, and rate-limited

Concern	Design
	telemetry.
Recovery	Workers reclaim pending stream entries after crashes.
Fallback	Neon read fallback if Redis fails; stale read-only display if Neon fails; local ESP32 safety rule if backend fails.

20.1 Local ESP32 fail-safe policy

- Fail-safe is intentionally narrow: it may turn devices OFF after a long verified vacancy but should not turn devices ON autonomously.
- Local rule requires fresh PIR operation and a configured maximum inactivity period.
- Local action is queued for audit synchronization when connectivity returns.
- Physical/manual switches always remain able to override the simulated logic.
- The dashboard labels local decisions distinctly from backend automation.

21. Deployment and CI/CD

Keep production topology simple: static dashboard, Bun API, Bun worker, Bun Discord bot, managed Redis, and Neon. The simulator may run as a separate demo service.

21.1 Deployment units

Unit	Deployment role
dashboard	Static Vite build served by a static host or the Bun API.
api	Long-running Bun service with HTTP and WebSocket support.
worker	Long-running Bun stream consumer and scheduler.
discord-bot	Long-running Gateway client or HTTP interactions service.
simulator	Optional long-running demo process, disabled in real mode.
redis	Managed Redis or container for local development.
neon	Managed Postgres with production, staging, and test branches.

21.2 CI pipeline

1. Install dependencies with locked Bun versions and bun.lock.
2. Run formatting, lint, and type checking.
3. Run unit and contract tests.
4. Create an isolated test database/branch and apply migrations.
5. Run integration and API tests.
6. Build dashboard and server applications.
7. Validate OpenAPI and WebSocket event schemas.
8. Run selected E2E tests against preview deployment.
9. Block merge on migration, type, test, or build failure.
10. Tag releases and record schema version.

21.3 Git and commit practice

- Use small feature branches and pull requests.
- Commit migrations with the code that requires them.
- Use conventional commit prefixes such as feat, fix, docs, test, refactor, and chore.
- Keep diagrams and documentation versioned beside code.

- Add screenshots and demo instructions to README.
- Never commit .env, bot tokens, database URLs, PIN hashes, or AI keys.

22. Implementation Roadmap

The roadmap protects the MVP while creating visible checkpoints for standout features.

Phase	Deliverables
Phase 0 - Foundation	Monorepo, shared contracts, platform PIN gate, Redis session handling, Neon migration, Redis connection, seed data, health endpoints, CI.
Phase 1 - Mandatory live monitoring	Simulator, device state, room totals, daily kWh, /bootstrap, WebSocket, basic dashboard.
Phase 2 - Mandatory alerts and Discord	After-hours and two-hour rules, active alerts, !status, !room, !usage, proactive alert channel.
Phase 3 - Digital twin and PIR	Top-view UI, animations, occupancy confidence, vacant-room waste, representative schematic.
Phase 4 - Safe control and automation	Control PIN, commands, timers, warning/cancel/snooze, inactivity shutdown, closing schedule, audit.
Phase 5 - Shared agentic AI	Read tools, dashboard chatbot, Discord /ask, action proposals, confirmations, consistent answers.
Phase 6 - Analytics and standout features	Charts, rankings, budgets, forecasts, savings, carbon, reports, presets, health, maintenance.
Phase 7 - Reliability and polish	PWA, public display, reconnect, stale state, failure tests, demo scenarios, documentation, video.

22.1 Scope priority

Priority	Scope
P0 - must finish	All case requirements, 15-device correctness, shared backend, live dashboard, mandatory alerts, Discord commands, diagrams, README, demo.
P1 - strongest differentiators	Digital twin, PIR occupancy, vacant-room alert, safe auto-shutdown, smart closing checklist, agentic chatbot, audit history.
P2 - valuable analytics	Historical charts, cost/waste, savings, rankings, peak/health/abnormal alerts, weekly comparison, bill forecast.
P3 - polish if time remains	PWA install, public display, PDF reports, carbon estimate, maintenance, local ESP32 fail-safe, advanced preferences.

23. Acceptance Criteria

A feature is complete only when its API, UI or integration behavior, persistence, tests, failure handling, and documentation are all present.

- Exactly 15 devices are present and no screen or API claims 18 devices.
- The dashboard, REST APIs, and WebSocket data remain inaccessible until a valid platform PIN session is established; logout, expiry, rate limiting, and temporary lockout behave correctly.
- Changing simulator state updates Neon/Redis, dashboard, power totals, timeline, and Discord answers consistently.
- Dashboard updates without refresh and recovers after WebSocket interruption.
- Both mandatory alerts trigger under accelerated demo time and include timestamps.
- PIR state transitions are visible and stale PIR never causes an unsafe vacancy shutdown.
- Protected actions require valid control authorization and explicit confirmation where specified.
- Every command and automation is idempotent and appears in audit history.
- Agentic AI uses tools for facts and cannot execute protected actions directly.

- Discord !status, !room, and !usage use live backend data.
- CSV/PDF report results match API analytics for the same interval.
- Health page reports component freshness and last known state during outages.
- Public repository setup works from .env.example and README.
- System diagram and representative hardware schematic are included without Mermaid.
- The three-minute demo can be performed deterministically without waiting for real time.

24. Three-Minute Demo Plan

The demo should prove synchronization and operational value, not merely show screens.

Time	Demonstration
0:00-0:20	Enter the platform PIN, then state the problem, 15-device scope, and shared-backend architecture.
0:20-0:50	Show digital twin, live watts, room totals, animated devices, and PIR occupancy.
0:50-1:15	Start after-hours scenario; show alert and live activity without refresh.
1:15-1:40	Run !status, !room work2, and !usage in Discord; show matching facts.
1:40-2:05	Ask dashboard AI: What should I check before leaving? Show smart closing checklist.
2:05-2:25	Propose room shutdown through AI, show explicit confirmation, warning countdown, and audit event.
2:25-2:42	Show savings/cost impact and one historical chart.
2:42-2:52	Show representative ESP32/PIR circuit and explain safe low-voltage simulation.
2:52-3:00	Close with GitHub structure, reliability, and future real-hardware path.

25. References

The implementation team should review the linked primary documentation when selecting exact package versions. Pin all production versions and keep the lockfile committed.

- **Hackathon case:** Hackathon Problem Statement (Preliminary Round) v1.2, pages 1-6.
- **React:** <https://react.dev/reference/react>
- **Vite:** <https://vite.dev/guide/>
- **Bun overview:** <https://bun.sh/docs>
- **Bun HTTP server:** <https://bun.sh/docs/runtime/http/server>
- **Bun WebSockets:** <https://bun.sh/docs/runtime/http/websockets>
- **Bun SQL:** <https://bun.sh/docs/runtime/sql>
- **Bun Redis:** <https://bun.sh/docs/runtime/redis>
- **Redis Pub/Sub:** <https://redis.io/docs/latest/develop/pubsub/>
- **Redis Streams:** <https://redis.io/docs/latest/develop/data-types/streams/>
- **Neon connection pooling:** <https://neon.com/docs/connect/connection-pooling>
- **Neon branching:** <https://neon.com/docs/introduction/branching>
- **Discord application commands:** <https://docs.discord.com/developers/interactions/application-commands>
- **Discord interactions:** <https://docs.discord.com/developers/interactions/receiving-and-responding>
- **Discord Gateway:** <https://docs.discord.com/developers/events/gateway>
- **OpenAI function calling:** <https://developers.openai.com/api/docs/guides/function-calling>
- **OpenAI structured outputs:** <https://developers.openai.com/api/docs/guides/structured-outputs>

Final architecture statement

OfficePulse is not a collection of separate demos. It is one event-driven energy platform: a single Bun domain backend, Neon history, Redis realtime coordination, a React digital twin, a shared agentic AI tool layer, and a Discord interface that all report and act on the same verified office state.